

Career Compass SG — Presentation Script

Module 1 Assignment · Group 7 · 10 minutes · 4 speakers

How to use this document. Each section has a **time budget**, the **script** (speak it, don't read it), and a **backup** box with the numbers to reach for if you are questioned. The bold sentences are the ones that must land — if you are running short, say only those.

Setup before you start: dashboard running at `http://localhost:8501` , browser on the **Market Overview** page, sidebar filters at their defaults (May 2023 – May 2024), notebook open in a second tab in case someone asks to see the cleaning code.

Timing plan

#	Section	Speaker	Time	Running total
1	Business case & objective	A	2:30	2:30
2	Process & data handling	B	3:30	6:00
3	Dashboard walkthrough (live demo)	C	3:00	9:00
4	Challenges, learnings & next steps	D	1:00	10:00

Discipline rule: whoever is on section 2 has the most material and is the most likely to overrun. If section 2 passes 6:30 on the clock, skip the cleaning table and go straight to "the finding that changed the design".

1 · Business case & objective — Speaker A · 2:30

(Slide: title + the four dimensions)

Everyone in this room has, at some point, looked at a job board and asked a question the job board cannot answer.

A job board tells you **what jobs exist right now**. It does not tell you **which of them is worth going after**. And that second question is the one that actually decides the next five years of your working life.

Our user is a Singapore job seeker or a mid-career switcher — someone standing at a fork, deciding which career track to aim at. Our secondary user is the career coach advising them.

Their decision is: which career track should I target next? And that turns on four things, which no job board ever shows you side by side:

- **Demand** — are there enough openings to realistically land one?
- **Pay** — what does this actually pay, at my level?
- **Competition** — how many people am I up against for each seat?
- **Accessibility** — can I get in with the experience I have *today*?

So we built **Career Compass SG**: it scores **215 career tracks** — 43 job categories across 5 seniority levels — on those four dimensions, from **1,044,597 real Singapore job postings** spanning October 2022 to May 2024.

Our success test is three minutes. Open the dashboard, and within three minutes you can say: *these are the three tracks I should target, this is what they pay, and this is how contested they are*.

One thing worth admitting up front: **this was not our first idea**. We started with a "top-paying jobs in Singapore" dashboard. The data killed it — pay, volume and competition turn out to be almost independent of each other, so ranking on any one of them is actively misleading. My colleague will show you the numbers that forced that change.

Backup — if asked "why not just use LinkedIn?" LinkedIn shows you *postings*. It never shows you the applicant-to-seat ratio, and it never compares a track you are in against one you are not. That comparison is the entire product.

2 · Process & data handling — Speaker B · 3:30

(Slide: the three-stage pipeline diagram)

Python and pandas, in three stages, deliberately separated so the dashboard never touches the raw CSV.

The raw file is **273 megabytes, 1.05 million rows**. We read it in **six chunks of 200,000 rows**, clean each chunk, then concatenate for the two rules that need the whole file — de-duplication and outlier capping. Output is Parquet. **The whole pipeline runs in 15 seconds**. The dashboard then reads Parquet, and a filter-plus-group-by over 1.77 million rows takes **six hundredths of a second**. That speed is why our filters recompute live instead of serving frozen numbers.

(Slide: cleaning decisions table — walk 3 rows, not 11)

On cleaning, I want to give you our **principle** rather than eleven rules, because the principle is the interesting part: **blank the untrustworthy field, keep the trustworthy row**. A posting with a broken salary still proves a job exists.

Three examples:

- **occupationId** was 100% empty — all 1.05 million rows. Dropped the column.
- **10,016 postings claimed a monthly salary above \$60,000 or below \$500**. Those are annual figures typed into a monthly field. We **blanked the salary but kept the posting**, because it still counts as demand.
- **We capped the top and bottom 1% of salaries rather than deleting them** — 19,647 rows. Deleting would have biased our demand counts; capping stops a thin tail of executive packages from steering every average.

(Slide: the two-line engagement chart — this is the centrepiece)

Now the finding that changed everything.

We plotted views and applications over time as a sanity check. We were not looking for a problem. Look at what happens at June 2023: **mean views per posting fall off a cliff, from about 111 to about 5**. The share of postings with **zero applications jumps from 18% to 79%**.

That is not Singapore losing interest in jobs. **This dataset is two extracts stitched together**. Everything from July 2023 onward was captured at posting time, so the counters never accumulated.

So here is our design decision. Demand and salary use all 1.04 million rows. But **every competition metric in this product is computed only on postings up to June 2023** — 203,702 of them — and it says so on the screen, every time.

If we had missed this, every career track would have looked uncontested, and our recommender would have confidently pointed people at the most crowded job markets in Singapore. That is the difference between a dashboard and a wrong dashboard.

(Slide: the three-column table — size vs pay vs competition)

And this is what killed the "top-paying jobs" idea:

Category	Postings	Median pay	Applicants per seat
Information Technology	140,866	\$6,500	4.8
Admin / Secretarial	117,854	\$2,900	5.7
F&B	73,731	\$3,301	1.0
Social Services	8,684	\$3,500	11.3

Competition varies by a factor of 16. Two categories with similar numbers of openings can give you completely different odds. Which is exactly why our score has four components and not one.

Backup — if asked about duplicates: zero genuine duplicate job ids. `.duplicated()` reported 289 in our sample, but all 289 were repeated *blanks*. We kept the de-duplication step as a safeguard and documented it as a no-op.

Backup — if asked about the categories column: it is a JSON array; a job carries 1.69 categories on average. So we build two tables — one row per posting for market totals, one row per posting-times-category for category analysis. Category shares overlap and don't sum to 100%. We actually shipped that bug to ourselves for an hour: the overview page reported 1.66 million postings instead of 1.04 million.

3 · Dashboard walkthrough — *Speaker C · 3:00 · LIVE*

Demo discipline: four clicks, not fourteen. Rehearse the exact path below. Do not free-roam.

Click 1 — Market Overview (~30 s)

One million postings, **\$3,850 median monthly salary**, 36% of them open to someone with a year of experience or less.

Notice two things we do everywhere. **We quote medians, not means** — the distribution is right-skewed, and you can see both lines on this histogram. And **the grey band on the timeline** marks the months where our data collection was still filling up, so nobody reads it as a hiring collapse.

Click 2 — Demand vs Competition (~50 s)

This is the analytical core. Every dot is a category. **Across is competition, up is volume**, bubble size is open seats, colour is pay.

Top left is where you want to be: lots of openings, few rivals. F&B, Customer Service, Sales — real volume, almost no queue. **Top right is the trap:** big, well-known, and crowded.

And this orange box is permanent — it tells you these numbers come from the reliable window only. We would rather show you a caveat than a confident wrong number.

Click 3 — Career Recommender (~80 s — *this is the product, spend the time*)

Now the actual product. I'll be a **career switcher with 3 years of experience who needs at least \$4,000 a month.**

(set the inputs)

Top of the list: **Information Technology at Mid level** — 69,000 postings, \$6,500 median, four applicants per seat, and **57% of those postings are open to someone with my three years.**

These bars are why it ranked there — you can see exactly how much each of the four components contributed. Nothing is hidden.

But watch this. *(drag "Easy for me to enter" to maximum, drop "High pay")*

The entire ranking changes — HR, Sales, Education at entry level. Because for someone who needs a job in six weeks rather than the best job in five years, **that is genuinely the better advice.** The score is a set of sliders, not a verdict.

Click 4 — Career Explorer (~20 s)

And if you don't believe your shortlist, every recommendation is auditable. Pick IT: the pay ladder by seniority, the real job titles being advertised, and who is posting them — which, note, we label **"Poster"** and not "Employer", because the top five are all recruitment agencies.

If the live demo fails: switch to the slides and narrate the same four screens from screenshots. Do not debug in front of the room.

4 · Challenges, learnings & next steps — Speaker D · 1:00

(Slide: three bullets)

Three things we take away from this.

One — plot every metric against time before you trust it. The June 2023 break was invisible in summary statistics. `.describe()` would never have shown it. It only appeared when we drew it, and it would have inverted our conclusions.

Two — when one row means two different things in two tables, make the difference structural. Our category explosion turned 1.04 million postings into 1.77 million rows, and for a while our headline number was 60% too high — and plausible enough that nobody would have questioned it. We fixed it with an integer job key that every market-level statistic has to pass through.

Three — a chart's job is one sentence. Our first opportunity map labelled all 43 categories and was unreadable. Nine labels made the point instantly.

Where this goes next. Skills are currently inferred from job *titles*, because the dataset has no skills field — parsing job descriptions would turn a keyword proxy into a real skills taxonomy. And the natural product step is a **saved profile with alerts**: store the user's weights and tell them when a track's score moves. That is the step from a dashboard to a system.

What we would say to our user in one sentence: the best-paying track and the easiest-to-enter track are almost never the same track — and now you can see both, and decide for yourself which one you need this year.

Anticipated questions

Question	Answer
"Why is competition only measured up to Jun 2023?"	The counters stop accumulating after that — 79% of later postings show exactly zero applications. Using them would make every track look uncontested. 203,702 postings is still an ample base.
"Isn't IT the obvious answer? Why build a tool?"	IT is the biggest <i>and</i> best-paying category — and its postings fell 11.7% in the last six months, the third-steepest decline of any category. The obvious answer is also the cooling one. That is exactly what a tool is for.
"How do you know these are real employers?"	We don't, and we say so. <code>postedCompany_name</code> is the <i>poster</i> ; the top five are all recruitment agencies. We label the column "Poster" and warn the user.
"Why medians everywhere?"	Skew of 1.95. The mean is \$4,674 against a \$3,850 median — the mean describes almost nobody.
"Your skill list is just keywords."	Correct, and it is labelled as such in the app. The dataset has no skills field, so we match titles against a 50-term dictionary. It measures <i>what employers advertise for</i> , which is what a job seeker has to match — but it is not a skills census.
"Why cap salaries instead of dropping outliers?"	Dropping the row would remove the posting from the demand count too. Capping keeps the demand signal honest while stopping 19,647 extreme values from steering every average.
"How is this different from a salary guide?"	A salary guide has one dimension. Ours has four, and lets the user weight them. Change the weights and you get a different, equally defensible answer — that's the point.
"Could this scale to a real product?"	The pipeline already runs the full million rows in 15 seconds and the dashboard reads pre-built Parquet. The missing pieces are a scheduled refresh, user accounts and a hosted deployment — not the analytics.

Rehearsal checklist

- [] Full run-through against a clock — under 10:00 including handovers
- [] Speaker C has rehearsed the **exact four clicks**, including dragging the sliders
- [] Dashboard is already running before you walk up; browser zoom at ~80% so charts fit
- [] Screenshot fallback slides exist for all four demo screens
- [] Everyone can answer the **Jun 2023 engagement question** — it is the most likely one
- [] Handover sentences agreed, e.g. "...and B will show you how we got there"
- [] Notebook open in a second tab for "can we see the cleaning code?"